

Reinforcement Learning for Formula 1 Race Strategy Under the 2026 Power Unit Regulations

---

System Name: SENNA
(Strategic Engine for Next Gen Race Navigation and Analytics),
a reinforcement learning system for Formula 1 race strategy

Name: Mohammed Saad
Student ID: 201959151
MSc Advanced Data Science and AI
University of Liverpool
Supervisor: Dr Frank Wolter
Module: COMP702

Data category A0 (publicly available, non-personal data). No participant categories apply at this stage. I confirm I will follow the School's ethical guidance throughout the project.

---

1 Statement of Ethical Compliance

This project falls under data category A0 (publicly available, non-personal data) for the FastF1 and OpenF1 telemetry and timing data used to build the simulation environment and the reward model. No participant categories apply at this stage, as no human data collection is currently planned.

If a usability evaluation with other students or staff is added later, it will be conducted under category C1 (low-risk, anonymous feedback from adult participants, with no vulnerable groups involved). I will consult my supervisor and update this statement before that activity begins. I confirm that I will follow the School's ethical guidance throughout the project.

2 Project Description

This project builds a reinforcement learning system for Formula 1 race strategy, named Senna, that decides when a car should pit and which tyre compound it should fit. Senna is delivered as a web application. A user picks a circuit and a set of race conditions, the system runs a trained agent against a field of rival cars, and it returns a recommended strategy along with the reasoning behind it.

The core of the system is a custom race environment built with Stable-Baselines3 and trained using Proximal Policy Optimisation (PPO). The environment models tyre degradation, pit loss, and the 2026 power unit rules, which raise the MGU-K output to around 350 kW, remove the MGU-H, and change how a car may deploy and recover electrical energy across a lap [1]. These rules are recent enough that no public strategy tool accounts for them yet, and closing that gap is the point of the project.

The trained agent is served through a Streamlit dashboard that reads from a PostgreSQL database of historical race data from 2022 onward, with 2026 race and regulatory data added as it becomes available. A full-stack version of this interface, with a FastAPI backend and a React frontend, is the project's stretch target. The Streamlit dashboard delivers a complete, working tool on its own, and the FastAPI and React build is the planned extension once the core is stable.

The part of the project I care about most is the explainability layer. Most strategy tools hand back a recommendation and stop, which leaves a strategist to either trust it or ignore it. Senna instead shows its working. Reward decomposition and counterfactual comparison let a user see why the agent made a given call, with feature attribution sitting on top as a more exploratory layer. This is what separates a usable strategy tool from a black box that happens to be accurate, and it is where I expect the real research contribution to sit.

3 Research Questions, Aims and Requirements

3.1 Research questions

The project is organised around three research questions.

- RQ1. Can a PPO agent learn pit and tyre-compound strategies that outperform scripted and search-based baselines under the 2026 energy regulations?
- RQ2. Does reward decomposition, combined with counterfactual comparison, make the agent's recommendations more interpretable and more trustworthy to a strategist?
- RQ3. Can a single trained policy generalise across different circuit archetypes without retraining for each track?

3.2 Aims

- Build a race environment that reflects the 2026 power unit regulations closely enough that the strategic decisions it produces, such as when to deploy electrical energy and when to pit, would read as sound advice to a real strategist.
- Train an RL agent that beats sensible scripted and search-based baselines across a representative set of circuits, and not only one track it has overfit to.

- Deliver a working dashboard where a non-technical user can run a race scenario and understand what the agent recommends and why, through a breakdown of the reward and a comparison against the alternatives the agent considered.

3.3 Requirements

I have split the requirements into must-have (MH) items, which define the minimum acceptable submission, and good-to-have (GH) items, which form the stretch scope. The split gives me a clear, pre-agreed point at which to stop adding scope if time runs short, without leaving the project incomplete.

ID	Requirement	Category
MH1	Unified data pipeline pulling FastF1 and OpenF1 data from 2022 onward, with available 2026 data, into a consistent PostgreSQL schema	Must-have
MH2	Custom Gymnasium environment modelling tyre degradation, pit loss, and the 2026 power unit energy rules	Must-have
MH3	PPO agent trained with Stable-Baselines3 that outperforms a fixed one-stop and a reactive rival baseline on at least three circuit types	Must-have
MH4	Streamlit dashboard, reading from PostgreSQL, that lets a user configure a race scenario and view the recommended strategy with a basic explanation	Must-have
MH5	Reward decomposition showing the contribution of each reward term (tyre, position, energy) to a given decision	Must-have
GH1	FastAPI backend exposing race setup, simulation, and result endpoints, backed by PostgreSQL	Good-to-have
GH2	React frontend providing the full-stack version of the dashboard	Good-to-have
GH3	Environment expansion: full 20-driver grid with Lap 0 compound selection, fuel-load modelling, and dynamic, ERS-constrained rivals	Good-to-have
GH4	Full flag simulation (yellow, red, safety car, virtual safety car) with per-circuit probability modelling	Good-to-have
GH5	Counterfactual comparison, and SHAP-style feature attribution via a surrogate model trained on the agent's policy outputs, showing the top alternative strategies and the factors behind each decision	Good-to-have
GH6	Multi-circuit generalisation: a single trained policy that performs across the full season	Good-to-have
GH7	Live race mode stepping through a real historical race lap by lap, showing where the agent's strategy would have diverged from what actually happened	Good-to-have

4 Key Literature and Background Reading

Reinforcement learning for race strategy is a small but growing area. Early work modelled pit strategy with discrete-event and Monte Carlo race simulations, layering in probabilistic events such as safety cars to make the simulated race more realistic [2]. This approach works well for assessing a fixed set of strategies, but it scales poorly once rival behaviour and energy management become state variables the model has to reason about directly. Recent work has moved towards deep RL, framing strategy as a sequential decision process with a large, partly continuous state space and rivals who react to the agent's choices, which is close to how I treat the problem here.

4.1 Algorithm selection

What pushed me towards PPO was the shape of the action space more than anything else. The pit and compound calls are discrete, but the energy decision is continuous, and not many algorithms handle both halves comfortably. DQN [3] would cover the discrete calls but has no natural way to set a continuous deployment level. SAC [4] and TD3 [5] are built for continuous control and would manage the energy side, yet they assume the whole action space is continuous and need awkward workarounds for the pit and compound decisions. A2C is the closest relative of PPO here, but I found it noisier and slower to settle.

PPO sits in the middle of all this. Its clipped objective stays stable across a mixed discrete and continuous policy without the long tuning runs the alternatives tend to demand [6], which counts for a lot when the schedule is fixed. It also has a mature implementation in Stable-Baselines3 [7], so I can spend my time on the environment and the reward, which is where the actual research lives, rather than fighting the optimiser.

4.2 Domain and software background

On the regulatory side, the FIA 2026 technical regulations set out the higher MGU-K output, the removal of the MGU-H, and the limits on how electrical energy may be deployed and harvested across a lap [1]. I treat these as primary source material, since there is no peer-reviewed literature yet on the strategic implications of these specific rules. I use existing strategy tools and write-ups from team strategists to sanity-check that the reward structure reflects real priorities such as tyre-cliff timing, undercut and overcut windows, and the cost of traffic.

On the software side, the FastF1 and OpenF1 API documentation defines what telemetry and timing data is available and at what resolution, which shapes what the environment can model [8], [9]. The Gymnasium environment API [10] and the Stable-Baselines3 PPO implementation [6], [7] guide the technical design of the RL component, including how I structure the custom reward function and observation space.

5 Development and Implementation Summary

The work splits into two halves that develop largely in parallel once the data pipeline is stable: the RL research component (environment, reward design, training, evaluation) and the application component (the dashboard). I build the RL component to a stable, testable state first and then build the dashboard against it. Putting an interface around a half-finished model usually means redoing the interface once the model's output settles, so I fix the model's contract early to avoid that.

The environment and training code are written in Python, using Gymnasium for the environment interface [10] and Stable-Baselines3 for PPO [7], since both are the standard tools for this kind of work and have the documentation and community support I need to debug quickly on a tight timeline. The must-have interface is a Streamlit dashboard that reads directly from PostgreSQL, which stores the processed race data and the simulation results. Streamlit keeps the core deliverable small and puts a usable tool in front of the model without a separate web service. The stretch version replaces it with a FastAPI backend [11] and a React frontend, giving the project a production-style, fully decoupled architecture if time allows.

I have organised development into discrete phases. Each phase has a defined, testable output, and I do not start the next phase until the current one is confirmed working. Phase 1, the data pipeline and PostgreSQL schema, is already complete. I use Git for version control with a branch-per-phase workflow, and I draft the dissertation alongside development from Phase 2 onward. Writing about a design decision while it is fresh keeps the record accurate and avoids a large reconstruction effort near the deadline, which is a common way these projects lose marks in the self-evaluation.

6 Data Sources

This project uses two publicly available data sources. FastF1 is a Python library that provides timing, telemetry, and session data for F1 races [8], and OpenF1 is a REST API that provides similar live and historical data [9]. Both are open source and free for non-commercial use, and neither requires an access agreement. The data spans seasons from 2022 onward and includes lap times, tyre compounds, pit stop timing, sector times, and weather conditions. Because the 2026 season is still in progress, I use the 2026 race and regulatory data that is available and rely on the complete 2022 to 2025 seasons as the historical base.

The data describes cars, lap times, and race events. Driver names appear only as part of the public sporting record, in the same way a published race classification does, so no anonymisation is required. I do not collect, generate, or process any other personal or sensitive data at any point in the pipeline.

7 Testing and Evaluation

Testing covers two layers. Unit and integration tests check that individual components behave correctly in isolation. The data pipeline produces a consistent schema across seasons, the environment's reward function returns expected values for known scenarios, and the dashboard surfaces the agent's output correctly for valid and invalid inputs. I run these continuously through development.

7.1 Baselines and metrics

I evaluate the trained agent against a set of baselines that bracket the strategy space. A fixed one-stop strategy and a reactive rival strategy give the basic comparison. A Monte Carlo strategy search [2] represents the classical approach that teams have historically used. A human-strategist heuristic encodes common rules of thumb, such as reacting to the tyre cliff and protecting track position. A random policy sits at the bottom as a sanity check that the agent has learned something real.

The Monte Carlo baseline needs care, because a weak search would flatter the agent. I define it by three things: the number of strategy rollouts it runs per decision, the planning horizon, which is how many laps ahead each rollout simulates, and the set of candidate strategies it samples from. To keep the comparison fair, I do not fix the rollout budget by hand. I raise it until the baseline's race time stops improving, record the budget at that plateau, and report it, so the agent is measured against a search that has been given enough simulations to be genuinely competitive rather than starved on purpose. The race simulation underneath the search is the same model used to train the agent, and I calibrate it against known historical race outcomes before any comparison, so both the agent and the search reason over a realistic race.

I score every strategy on four metrics: total race time, finishing position against the field, energy efficiency measured against the deployment and harvesting limits the 2026 rules impose, and pit-stop efficiency, which weighs the time lost in the pit lane against the time gained on track. Each metric exposes a different aspect of strategy quality, so together they show where the agent does well and where it falls short.

7.2 Targeted experiments

Two experiments target the research questions directly, and the more important of the two is the regulation comparison. I run the same circuits under the 2025 and the 2026 energy rules and measure how far the recommended strategy shifts between them. If the new rules genuinely change what counts as the best strategy, this is where it should show, and it is the part of the evaluation that speaks most directly to RQ1.

Running alongside it is an ablation on the reward. By switching off the tyre, position, and energy terms one at a time, I can see whether each term actually pulls its weight, and whether the agent's behaviour comes from the factors I intended rather than from a quirk of the reward shaping.

7.3 Interpretability and generalisation

For RQ2, the explainability methods are assessed on their own terms. Reward decomposition and counterfactual comparison are the core methods, and each gives a different view of why the agent chose a strategy. SHAP-style feature attribution is a stretch addition, applied to a surrogate model trained on the agent's policy outputs rather than to the PPO policy directly, since SHAP does not transfer cleanly to a policy network [12]. The optional usability check described in the ethics section gauges whether these views actually help a reader trust and understand the recommendation.

For RQ3, I evaluate the agent across the three circuit archetypes to confirm it generalises across circuit types and has not overfit to one track. If the multi-circuit generalisation item (GH6) is reached, the evaluation extends across the full season.

8 Project Ethics and Human Participants

This project does not currently involve human participants. No requirements-gathering exercise or user evaluation with other people is planned at this stage, and no personal data of any kind is collected. I have confirmed with my supervisor that this section genuinely does not apply to the current scope.

If a small usability check with other students is added later to assess how understandable the strategy output is (for example, asking a handful of people whether the reward decomposition view makes sense), it would run under category C1, with participation voluntary, feedback anonymised at the point of collection, and no data retained that could identify a participant. I would not add this without first confirming the change with my supervisor and updating this statement.

9 BCS Project Criteria

This section maps the project deliverables and development method onto the six BCS honours-project outcomes.

9.1 Systematic understanding and critical awareness at the forefront of the discipline

The forefront here is the regulation change itself. Nobody has published on how the 2026 rules reshape strategy, so the reward model is built straight from the technical regulations [1], and the regulation-comparison experiment puts a number on the strategy shift they cause. That experiment is the clearest evidence that the project is working at the current edge of the area.

9.2 Comprehensive understanding of applicable techniques

On technique, the work rests on Markov Decision Process framing, PPO, and a custom Gymnasium environment. Section 4.1 sets out why PPO, and not DQN, SAC, TD3, or A2C, is the right fit for an action space that mixes discrete and continuous decisions.

9.3 Originality in the application of knowledge

The original part is not the PPO agent, which is fairly standard now, but the pairing of a regulation-aware reward model with an explainability layer. Reward decomposition and counterfactual comparison let a strategist see the reasoning behind a call, where most tools simply return an answer and leave the reasoning hidden.

9.4 Handling complex issues and communicating to mixed audiences

Two audiences pull in different directions. The modelling has to cope with incomplete data, because no full 2026 season exists yet, and the dashboard has to make sense to someone with no machine learning background. The explainability work is what holds those two demands together.

9.5 Self-direction and professional-level autonomy

I am running this on my own, with supervision for direction rather than instruction. The phasing, the must-have and good-to-have split, and the choice to build the model before the interface are all my own calls, made to keep an ambitious scope inside a fixed deadline.

9.6 Critical self-evaluation of the process

Because I write the dissertation as I build, the record keeps the decisions that did not work alongside the ones that did. That makes the final self-evaluation an honest account rather than something reconstructed once the result is already known.

10 UI/UX Mockup

The interface is built around a single core screen. A race configuration panel on the left handles circuit selection, weather, and starting grid position. A results view on the right shows the recommended strategy as a timeline across the race distance, with pit stops and compound changes marked. Below this, a reward decomposition panel explains why that strategy scored as it did, and a comparison panel lists the next best alternatives the agent considered.

Figures 1 and 2 are placeholders for screenshots of the working dashboard. The first version of the layout is kept deliberately simple. The explainability panels are the part most likely to change once real model output is available to design against, so I treat them as a starting point.

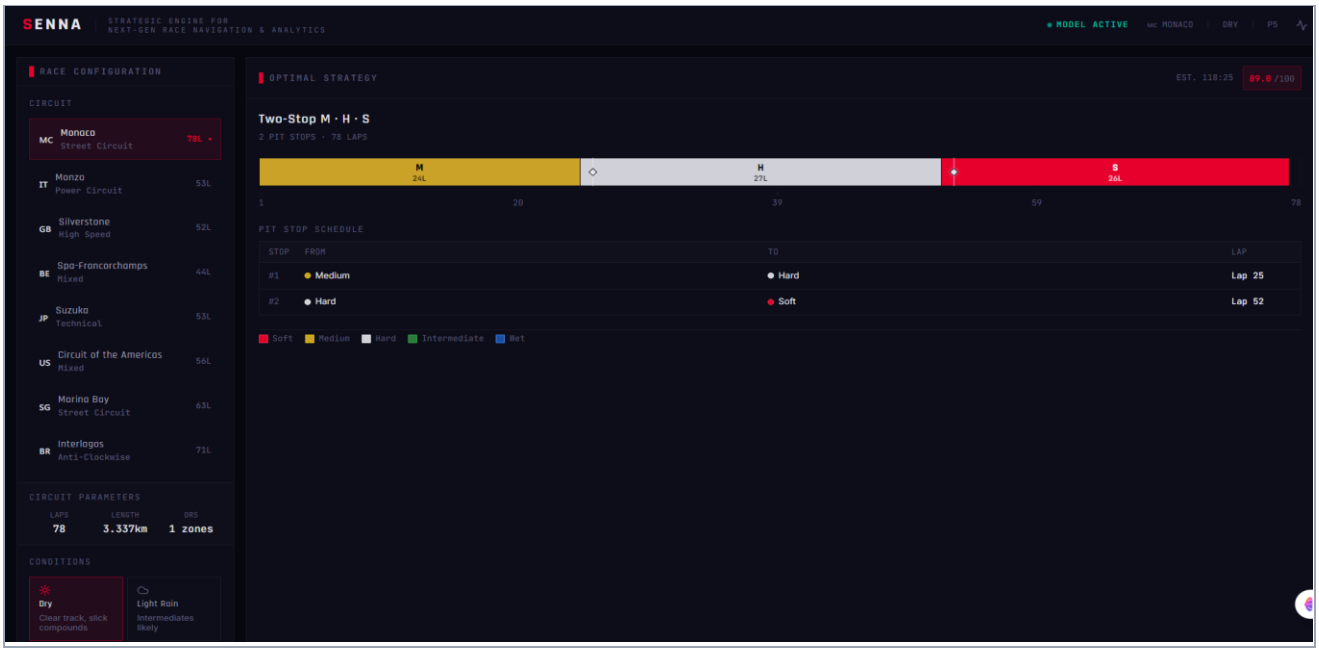


Figure 1. Configuration panel and strategy timeline.

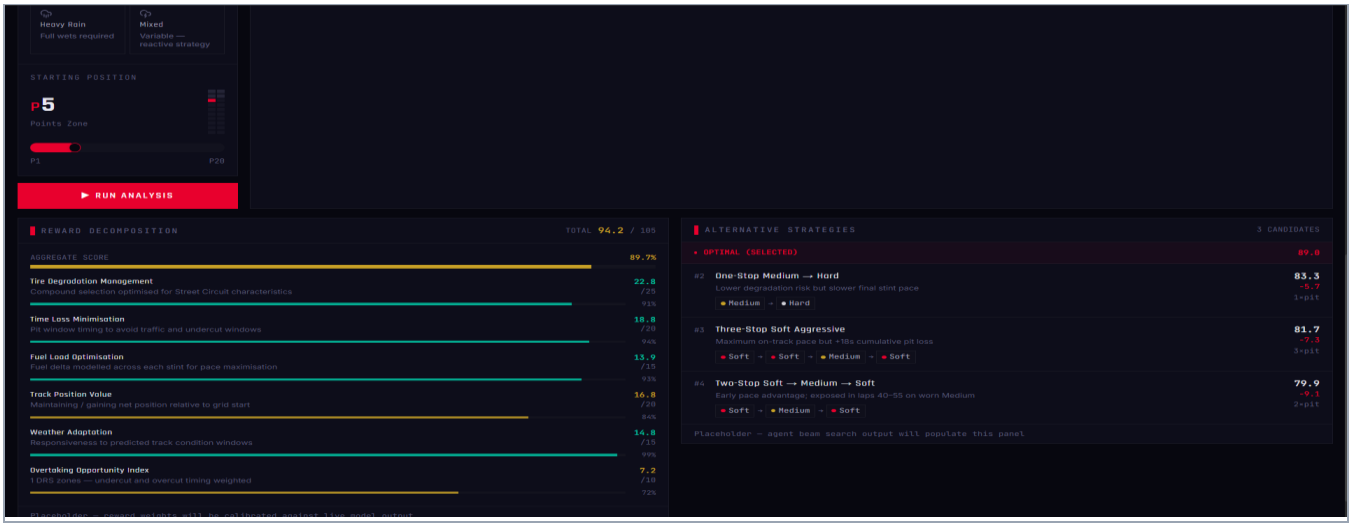
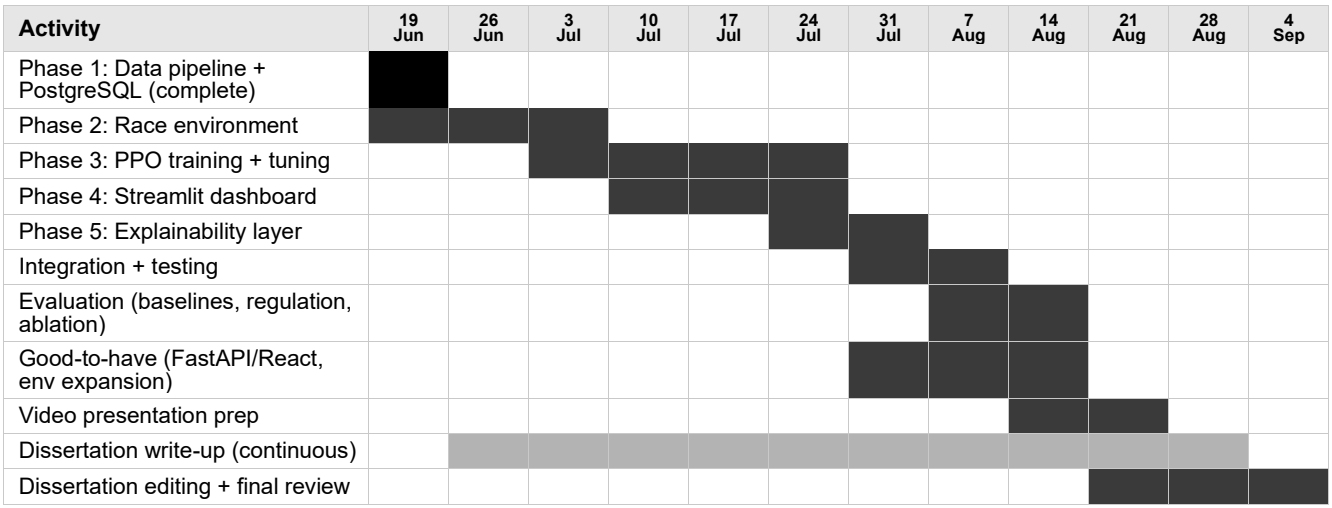


Figure 2. Reward decomposition and counterfactual comparison.

11 Project Plan

The plan below runs from the proposal deadline to dissertation submission. Phase 1, the data pipeline and PostgreSQL schema, is already complete and is shown at the start of the chart for context. Dissertation writing runs as a continuous activity alongside development, since writing about a phase while it is fresh is faster and more accurate than reconstructing it later. Where two phases overlap, this reflects genuine room to work on both at once. For example, the dashboard layout can begin against a placeholder response before the agent is fully trained.



12 Risks and Contingency Plans

Risk	Contingency	Likelihood	Impact
Good-to-have scope (full grid, fuel modelling, flag simulation, multi-circuit generalisation) takes longer than planned and threatens the must-have deadline	The MH and GH split exists for this reason. GH items are dropped in reverse priority order (GH7 first) without affecting the core submission.	Medium	Medium
FastF1 or OpenF1 changes, rate-limits, or has gaps in 2026 data, since the season is still in progress	Phase 1 caches all pulled data locally in PostgreSQL, so training does not depend on repeated live API access. The 2022-2025 seasons give a complete fallback if 2026 data is incomplete.	Low	Medium
Hardware or software failure: corrupted environment, lost training run, machine failure	All code is version-controlled in Git with regular commits. Model checkpoints and the database are backed up separately to cloud storage, not only locally.	Low	High
The stretch full-stack build (FastAPI + React) takes longer to integrate against the model than planned	The Streamlit dashboard already delivers a complete tool, so the stretch can be dropped without affecting the core. The model-output contract is defined early using a placeholder response so the build can proceed against a stub.	Medium	Low
Running out of time before the dissertation deadline given the breadth of the project	The dissertation is written continuously alongside development, so no large write-up phase competes with final development work near the deadline.	Medium	High

References

[1] Fédération Internationale de l'Automobile, "2026 Formula 1 Technical Regulations," FIA, 2026.

[2] A. Heilmeier, M. Graf, J. Betz, and M. Lienkamp, "Application of Monte Carlo methods to consider probabilistic effects in a race simulation for circuit motorsport," Applied Sciences, vol. 10, no. 12, art. 4229, 2020.

[3] V. Mnih, K. Kavukcuoglu, D. Silver, et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529-533, 2015.

[4] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in Proc. 35th Int. Conf. Machine Learning (ICML), 2018, pp. 1861-1870.

[5] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in Proc. 35th Int. Conf. Machine Learning (ICML), 2018, pp. 1587-1596.

[6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[7] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," Journal of Machine Learning Research, vol. 22, no. 268, pp. 1-8, 2021.

[8] FastF1, "FastF1 documentation." [Online]. Available: https://docs.fastf1.dev/ (accessed Jun. 2026).

[9] OpenF1, "OpenF1 API documentation." [Online]. Available: https://openf1.org/ (accessed Jun. 2026).

[10] M. Towers et al., "Gymnasium documentation." [Online]. Available: https://gymnasium.farama.org/ (accessed Jun. 2026).

[11] FastAPI, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com/ (accessed Jun. 2026).

[12] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 4765-4774.